

Para este proyecto, vamos a construir una **"App de Rastreo y Seguridad para Repartidores (Ruta Segura)"**. Esta aplicación simula perfectamente un sistema de entregas de última milla (tipo Uber Eats o Mercado Libre) y combina de forma orgánica 3 de los módulos que ya revisamos:

1. **GPS (Geolocalización Continua)**: Para obtener la posición en tiempo real.
2. **Mapas Libres (flutter_map)**: Para renderizar la ruta del repartidor en un mapa interactivo gratuito.
3. **Sensores Biométricos (local_auth)**: Como mecanismo de seguridad para "Confirmar la Entrega" del paquete mediante huella o rostro, evitando fraudes.

Proyecto Integrador: Sistema de Rastreo de Envíos y Confirmación Biométrica

Este proyecto final busca la integración de múltiples capas de hardware (sensores de ubicación y seguridad biométrica) combinados con un sistema de información geográfica (SIG) de código abierto, estructurado bajo una arquitectura limpia y reactiva.

1. Arquitectura y Flujo del Proyecto

El objetivo del software es simular la jornada de un operador de reparto.

1. La aplicación inicia mapeando la ubicación del usuario mediante el chip **GPS** y centrando la cartografía digital (**OpenStreetMap**).
2. El sistema mantiene un monitoreo en tiempo real del movimiento del repartidor, actualizando un marcador dinámico en el mapa.
3. Al llegar al destino simulado, el operador debe presionar un botón de "Finalizar Entrega", el cual dispara un guardián de seguridad que bloquea la pantalla hasta que el usuario valide su identidad mediante el **sensor biométrico** del teléfono.

Estructura de Carpetas del Proyecto Integrador

Para mantener el orden que hemos trabajado en clase, crearemos una característica (**feature**) dedicada para el proyecto integrador llamada **delivery_tracker**:

```
mi_app_sensores/  
├── pubspec.yaml  
└── lib/  
    ├── main.dart  
    ├── core/  
    │   ├── theme/  
    │   └── theme.dart  
    └── features/  
        ├── home/  
        └── delivery_tracker/      <-- NUEVA CARPETA DEL PROYECTO INTEGRADOR  
            └── screens/
```

└─ tracker_screen.dart

Configuración del `pubspec.yaml`

Asegúrate de que tus alumnos tengan estas tres dependencias clave en su versión estable:

```
dependencies:  
  flutter:  
    sdk: flutter  
  geolocator: ^13.0.0  
  flutter_map: ^7.0.2  
  latlong2: ^0.9.1  
  local_auth: ^2.3.0
```

2. Código Fuente Completo y Comentado

Archivo `lib/features/delivery_tracker/screens/tracker_screen.dart`

```
import 'package:flutter/material.dart';  
import 'package:flutter/services.dart';  
import 'package:flutter_map/flutter_map.dart';  
import 'package:latlong2/latlong2.dart';  
import 'package:geolocator/geolocator.dart';  
import 'package:local_auth/local_auth.dart';  
import 'dart:async';  
  
class TrackerScreen extends StatefulWidget {  
  const TrackerScreen({super.key});  
  
  @override  
  State<TrackerScreen> createState() => _TrackerScreenState();  
}  
  
class _TrackerScreenState extends State<TrackerScreen> {  
  // Inicializadores de controladores de Hardware  
  final MapController _mapController = MapController();  
  final LocalAuthentication _auth = LocalAuthentication();  
  StreamSubscription<Position>? _gpsStreamSubscription;  
  
  // Estados de la UI  
  LatLng _currentPosition = const LatLng(19.4326, -99.1332); // CDMX por defecto  
  bool _isGpsReady = false;  
  bool _isPackageDelivered = false;  
  String _statusMessage = "Enganchando satélites GPS...";  
  
  @override
```

```
void initState() {
  super.initState();
  _initIntegratorService();
}

// 1. Inicialización e interconexión de servicios de localización
Future<void> _initIntegratorService() async {
  bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) {
    setState(() { _statusMessage = "Activa el interruptor de GPS en tu
teléfono."; });
    return;
  }

  LocationPermission permission = await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) {
      setState(() { _statusMessage = "Permiso de ubicación denegado."; });
      return;
    }
  }

  try {
    // Capturar la posición actual inicial
    Position position = await Geolocator.getCurrentPosition(
      locationSettings: const LocationSettings(accuracy: LocationAccuracy.high)
    );

    setState(() {
      _currentPosition = LatLng(position.latitude, position.longitude);
      _isGpsReady = true;
      _statusMessage = "Ruta de entrega activa";
    });

    _mapController.move(_currentPosition, 16.0);
    _startLiveTracking();

  } catch (e) {
    setState(() { _statusMessage = "Error de enlace de hardware: $e"; });
  }
}

// 2. Transmisión reactiva de coordenadas del GPS al Mapa
void _startLiveTracking() {
  const LocationSettings settings = LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 2, // Actualiza la UI cada 2 metros de movimiento
  );

  _gpsStreamSubscription = Geolocator.getPositionStream(locationSettings:
settings)
    .listen((Position position) {
      if (!mounted) return;
    });
}
```

```

        setState(() {
          _currentPosition = LatLng(position.latitude, position.longitude);
        });
        // Desplazamiento suave de la cámara del mapa persiguiendo al marcador
        _mapController.move(_currentPosition, _mapController.camera.zoom);
      });
    }

    // 3. Orquestador de Seguridad Biométrica para Cierre de Proceso
    Future<void> _confirmDeliveryBiometrically() async {
      // Comprobar primero si el dispositivo tiene capacidades biométricas activas
      bool canAuthenticate = await _auth.canCheckBiometrics || await
      _auth.isDeviceSupported();

      if (!canAuthenticate) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Código de respaldo: No hay sensores
biométricos configurados.')),
        );
        return;
      }

      try {
        // Disparar la validación nativa criptográfica
        bool authenticated = await _auth.authenticate(
          localizedReason: 'Verifica tu identidad con tu huella o rostro para firmar
la entrega.',
          options: const AuthenticationOptions(
            stickyAuth: true, // Protege el flujo si la app cambia de foco
            biometricOnly: false, // Permite usar el PIN del teléfono como respaldo
seguro
          ),
        );

        if (authenticated) {
          setState(() {
            _isPackageDelivered = true;
            _statusMessage = "Entrega Finalizada Exitosamente";
          });
          _gpsStreamSubscription?.cancel(); // Apagar el GPS ya que la ruta concluyó
        }
      } on PlatformException catch (e) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Error de seguridad: ${e.message}')),
        );
      }
    }

    @override
    void dispose() {
      _gpsStreamSubscription?.cancel();
      _mapController.dispose();
      super.dispose();
    }
  }

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Proyecto Integrador: Logística Segura')),
    body: Stack(
      children: [
        // Capa de Fondo: Cartografía e hilos de renderizado de mapas
        FlutterMap(
          mapController: _mapController,
          options: MapOptions(
            initialCenter: _currentPosition,
            initialZoom: 15.0,
          ),
          children: [
            TileLayer(
              urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
              userAgentPackageName: 'com.example.mi_app_sensores',
            ),
            if (_isGpsReady)
              MarkerLayer(
                markers: [
                  Marker(
                    point: _currentPosition,
                    width: 60,
                    height: 60,
                    alignment: Alignment.topCenter,
                    child: Icon(
                      Icons.local_shipping,
                      color: _isPackageDelivered ? Colors.green :
Colors.blueAccent,
                      size: 45,
                    ),
                  ),
                ],
              ),
          ],
        ),

        // Capa de Estado Superior: Indicador de Telemetría
        Positioned(
          top: 15,
          left: 15,
          right: 15,
          child: Card(
            color: Colors.white.withOpacity(0.95),
            elevation: 6,
            child: Padding(
              padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 16),
              child: Row(
                children: [
                  Icon(
                    _isPackageDelivered ? Icons.check_circle :
Icons.radio_button_checked,

```

6 / 8

```

        style: const TextStyle(fontSize: 18, fontWeight:
FontWeight.bold),
        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 8),
      Text(
        _isPackageDelivered
          ? 'Token criptográfico guardado en hardware seguro.'
          : 'Destino: Ubicación actual del operador (Validación de
cercanía).',
        style: const TextStyle(color: Colors.grey, fontSize: 13),
        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 16),
      ElevatedButton.icon(
        onPressed: _isPackageDelivered ? null :
_confirmDeliveryBiometrically,
        style: ElevatedButton.styleFrom(
          backgroundColor: _isPackageDelivered ? Colors.grey :
Colors.green,
          foregroundColor: Colors.white,
          padding: const EdgeInsets.symmetric(vertical: 15),
        ),
        icon: Icon(_isPackageDelivered ? Icons.done_all :
Icons.fingerprint),
        label: Text(
          _isPackageDelivered ? 'ENTREGA CONCLUIDA' : 'CONFIRMAR
ENTREGA CON BIOMETRÍA',
          style: const TextStyle(fontSize: 15, fontWeight:
FontWeight.bold),
        ),
      ),
    ],
  ),
),
),
),
),
],
),
);
}
}

```

Registro en la Consola Principal

Para puedan acceder a este proyecto, enlaza la pantalla en

[lib/features/home/screens/home_screen.dart](#):

```

// 1. Importar la pantalla del proyecto integrador al inicio:
import '../delivery_tracker/screens/tracker_screen.dart';

```

```
// 2. Agregar un botón de destaque completo en el GridView o debajo de él:  
_SensorCard(  
  title: 'PROYECTO INTEGRADOR: Ruta Segura',  
  icon: Icons.local_shipping,  
  color: Colors.green, // Destacado en verde de éxito comercial  
  onTap: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) => const TrackerScreen()),  
    );  
  },  
),
```